MASARYK
UNIVERSITY

FACULTY OF INFORMATICS

Musikk: A Modern, Social-First Music Streaming Platform

Bachelor's Thesis

KIRILL VOROZHTSOV

Brno, Fall 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Musikk: A Modern, Social-First Music Streaming Platform

Bachelor's Thesis

KIRILL VOROZHTSOV

Advisor: Mgr. Luděk Bártek, Ph.D

Brno, Fall 2025


FACULTAS ARTIS INFORMATICÆ
UNIVERSITAS MASARYKIANA

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, the following AI tools were used:

- **ChatGPT**: Used for debugging and code improvements, as well as LaTeX formatting.
- **Grammarly**: Used for text correction.

Kirill Vorozhtsov

Advisor: Mgr. Luděk Bártek, Ph.D

Acknowledgements

I would like to thank Mgr. Luděk Bártek, Ph.D., for his time and the valuable advice he provided for this thesis.

Abstract

This bachelor's thesis aims to develop a web-based music streaming platform focused on social interactions between users.

The theoretical part presents a survey that analyzes people's music listening habits, showing that such a platform could attract a solid user base. It then compares existing music-related services with built-in social features and derives functional and non-functional requirements for the proposed system. Then, based on these requirements, suitable technologies and architectural patterns are selected.

The practical part covers the implementation details of the platform, including data model, audio processing, API design, and real-time synchronization.

The result is a fully functional and deployed application.

Keywords

Audio Streaming, Python, JavaScript, MPEG-DASH, HLS, WebSockets

Contents

List of Tables

List of Figures

1 Introduction

Music streaming has been steadily growing in popularity each year for the past decade. [1] And this is to be expected; numerous studies have demonstrated how important music is for people. For instance, Rentfrow [2] suggests that music can influence mood, change self-perception, and help with social bonding. In addition, the rise of the clubbing scene, music festivals, and other social music-related activities clearly indicates the important social role of music. [3]

However, despite this social significance, it is surprising that features that facilitate social interactions are still not widely implemented in the existing music streaming platforms, as will be discussed in **Chapter 3**.

The goal of this thesis is to design a music-centric platform that provides capabilities for collaboration and social interaction around music. This work is divided into the following **six** chapters:

- **Chapter 2** Presents the outcomes of a survey illustrating how people consume music, how prevalent it is in social interactions, and why this type of platform can be relevant.
- **Chapter 3** Explores what existing streaming platforms and other music-related services already offer.
- **Chapter 4** Outlines the functional and non-functional criteria for the application.
- **Chapter 5** Describes the choices of technologies that are used by the application.
- **Chapter ??** Explains the development process and implementation details.
- **Chapter ??** Summarizes the results of the work.

2 Music Consumption Survey

In order to better support the choice of topic and demonstrate that a music streaming platform centered on social interactions could be a relevant service, a brief survey was conducted. It examined individual music listening and discovery habits, platform usage patterns, music-related interactions between people, and attitudes towards potential social features on the platform.

2.1 Methods

The service chosen for the questionnaire was Google Forms [4], as it provides a simple interface for survey creation, allows for easy sharing of the form, and supports extracting results as CSV files.

The questionnaire itself consists of 15 questions. Most of them are multiple-choice and closed-ended, with some allowing a custom answer. All custom answers are grouped under the "other" category in the provided figures. Answers in their original form can be found in the full survey.

The complete questionnaire is provided in Appendix A.1 and online.[1]

2.2 Results

This section presents the survey outcomes and analyzes their significance. Not all questions is discussed; for example, specific questions dedicated to music discovery are omitted.

A total of 119 people participated, with the majority being from Russian-speaking countries. Most respondents were in the 18 to 30 age group, with the exact distribution shown in Figure 2.1.

1. `https://docs.google.com/forms/d/1vhhAu_SfuHV4xy6JoDaRsM5uCElYn_csTmN8u4ZlpHc`

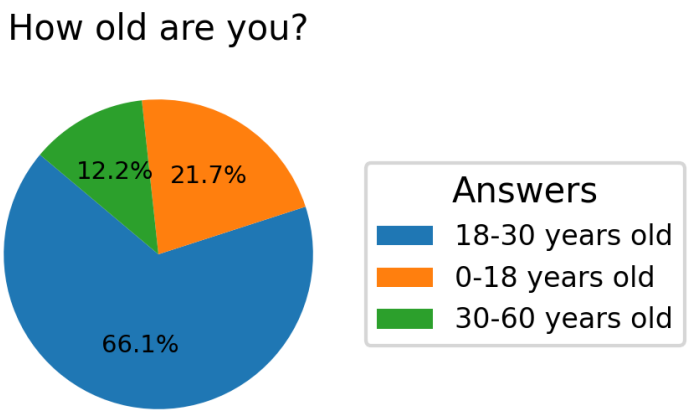

Figure 2.1: Age Distribution of Respondents

2.2.1 Engagement

Firstly, it was necessary to determine whether respondents listen to music frequently and whether they use streaming platforms for this purpose.

Over 50% of respondents reported listening to more than 500 songs per month, i.e., 16 songs a day on average (Figure 2.2), and nearly 80% stated that they listen to music daily (Figure 2.3).

The numbers clearly indicate that people have a significant interest in music, and the next logical step was to determine how the audio content is predominantly accessed.

2.2.2 Listening Methods

As shown in Figure 2.4, the proportion of respondents using streaming platforms accross all age groups is approaching 100%, with slightly higher numbers in the younger age groups. Although downloaded files and physical media are still used, especially among people aged 30-60, they are usually only a secondary option next to streaming solutions.

The question dedicated to the popularity of individual platforms has shown that Spotify is the most used one, which aligns with global statistics [5]. However, Yandex Music, coming in second place, de-

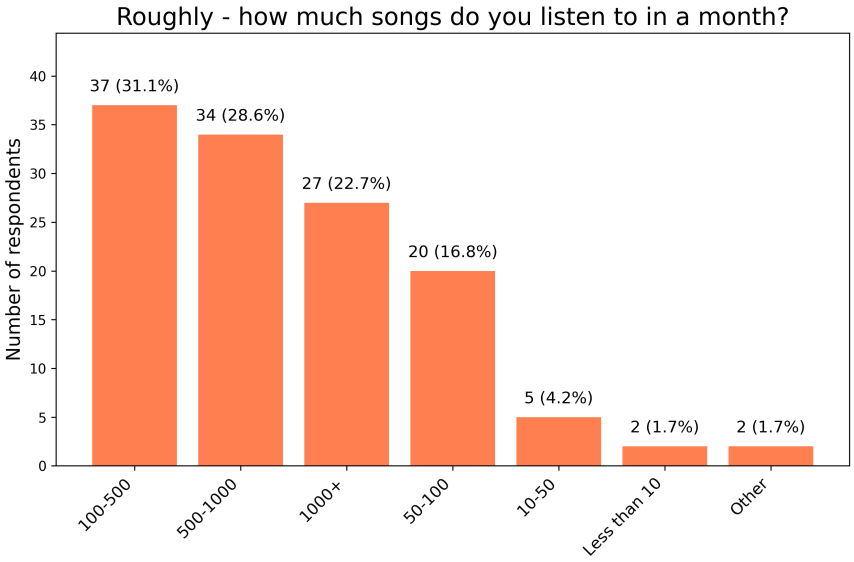

Figure 2.2: Monthly Song Count per Respondent

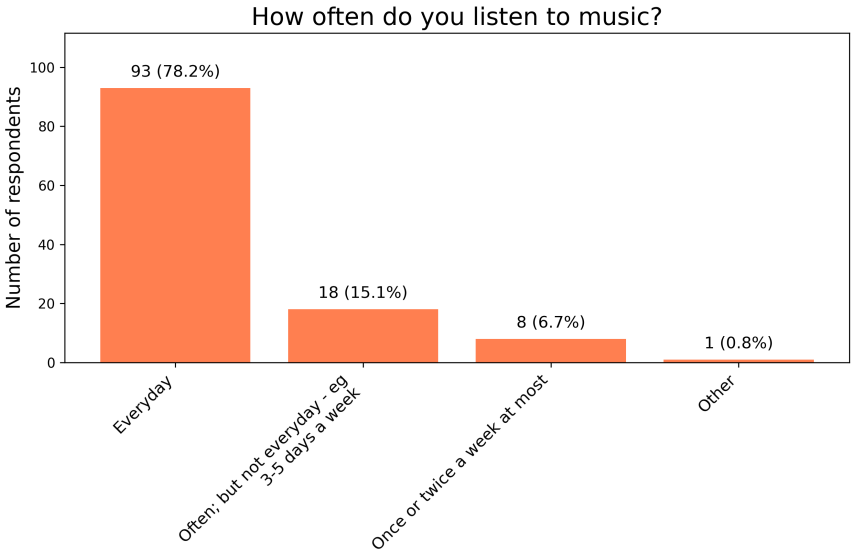

Figure 2.3: Listening Frequency of Respondents

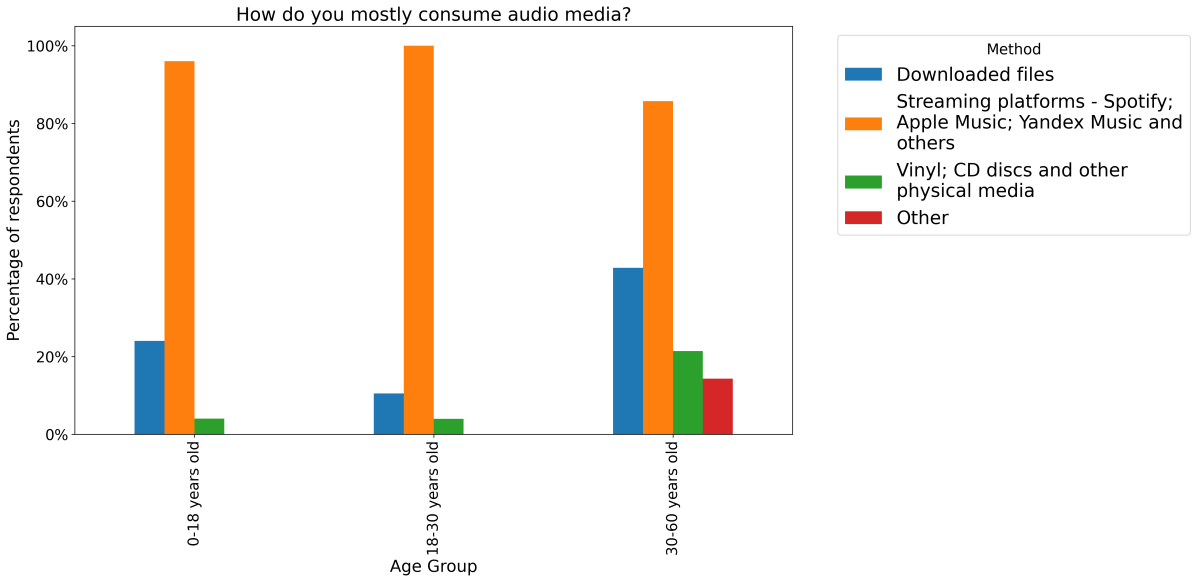

Figure 2.4: Audio Media Consumption by Age Group

serves further explanation. As mentioned previously, most of the respondents are from Russian-speaking countries, specifically Russia. With many Western companies leaving its market in 2022, including music streaming services, most users have moved to locally available products: Yandex Music, VK Music, and Zvuk. Another notable point is the absence of other popular region-specific platforms, such as Amazon Music for the US market, QQ Music for the Chinese market, and JioSaavn for the Indian market, as all of the respondents were based in the European part of the world. The complete statistics are shown in Figure 2.5.

The survey showed that streaming services are indeed widely used. The next important step was to analyze the regularity of music-centered social interactions.

2.2.3 Social Interactions

The results of Question 10 indicate that nearly all respondents (99.2%) reported listening to music with others, primarily in offline settings such as gatherings or car rides (Figure 2.6). Online collaborative listening methods, although less common, were also mentioned by a

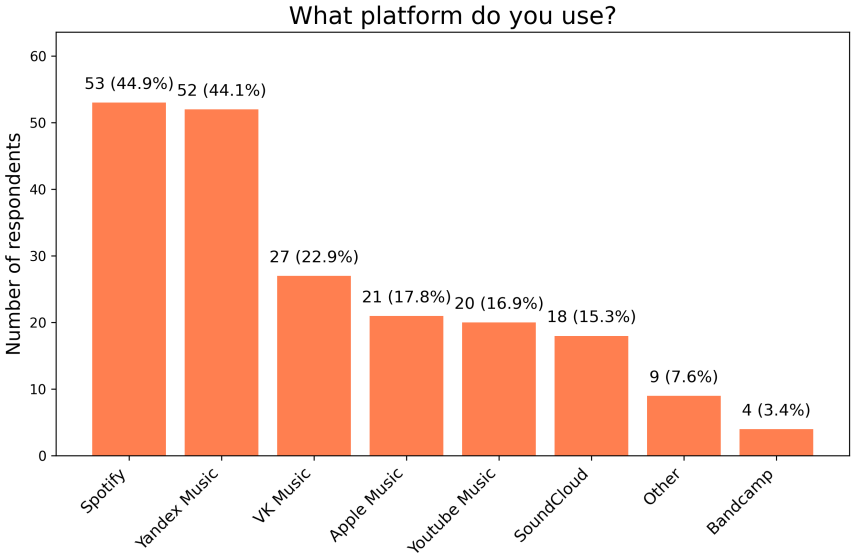

Figure 2.5: Streaming Platforms Popularity

quarter of respondents, indicating that digital solutions for shared listening are used, albeit not as prevalently as in-person scenarios.

Question 11 (Figure 2.7) examines the frequency of these interactions: the results show that a significant portion of respondents engage in shared listening regularly. Around 60% reported doing so at least a few times per month, with a smaller group indicating almost daily interactions. This supports the previously mentioned idea in **Chapter 1** that music consumption is a common social activity.

The next question is phrased as follows: "How often do you/your friends share/discuss music?". While the previous question focused on real-time in-person collective listening, this one highlights more intentional and in-depth musical interactions, such as sharing songs, recommending music, or discussing it. Music, in that case, is not just the background but the primary purpose of engagement. The same trend as before can be observed in the responses: over 70% (Figure 2.8) indicated that they share music at least several times a month, with many doing so weekly or more often. This further emphasizes the active social role of music.

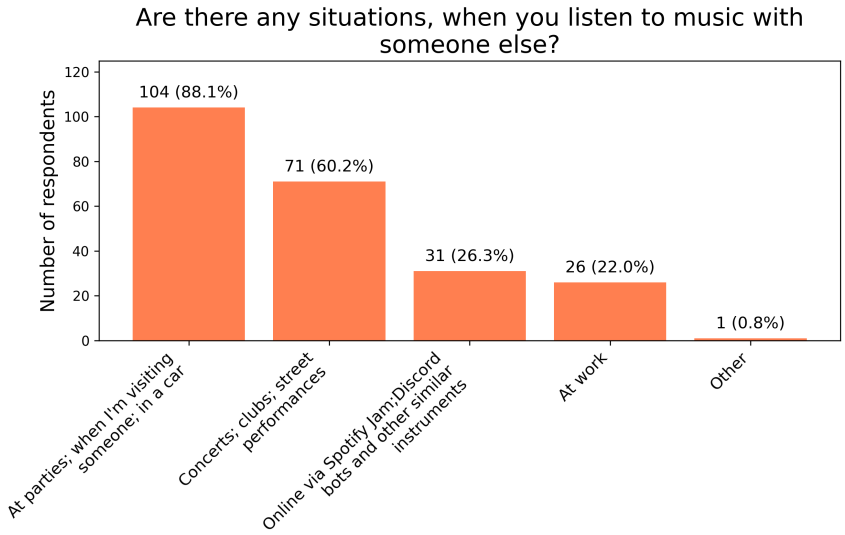

Figure 2.6: Social Interaction Methods

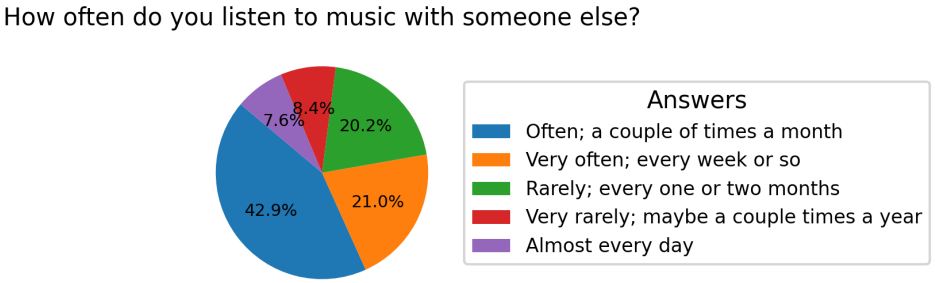

Figure 2.7: Social Interactions Frequency

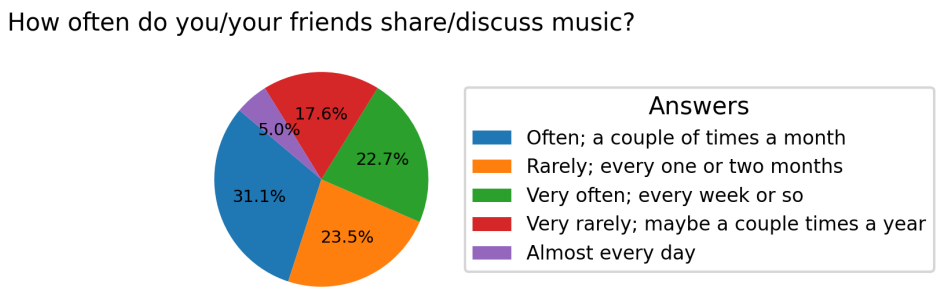

Figure 2.8: Music Sharing Frequency

Question 13, "How does that happen?" (in terms of music sharing), asks about the specific ways people share music. Many respondents (Figure 2.9) stated that they typically send links from streaming platforms through external messaging apps. While this shows an apparent demand for sharing music online, it also reveals a gap among platforms, as these interactions remain fragmented. Hence, enabling such exchanges directly within the streaming app could provide a smoother and more uniform experience.

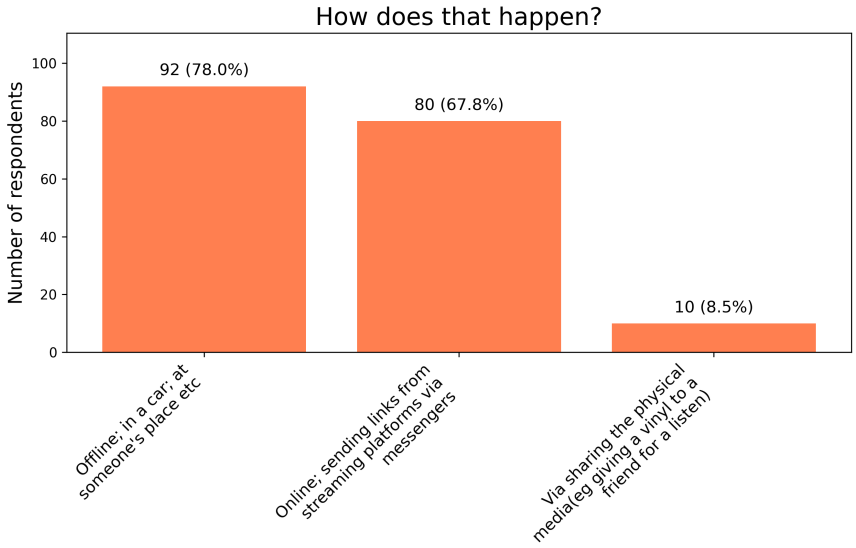

Figure 2.9: Music Sharing Methods

Responses to the question "Would you say that you know a lot of people with similar music taste?" were mixed, with a near-even split. This suggests that while some users already have a social circle with similar music taste, many do not. In addition, when asked "Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?", the majority answered positively.

This data further support the idea that social discovery features could be useful within a streaming platform. Moreover, over 60% of participants stated in Question 15 that they would use additional social features if available on a streaming platform.

2.3 Summary

Overall, the results indicate that people listen to music frequently, with streaming services being their primary medium of accessing it. The data also suggest that music plays an important role in everyday life and often appears in social situations.

Yet, existing platforms only partly support interaction between users. This makes the idea of a music streaming service with built-in social features both relevant and likely to address real user needs.

3 Existing Platforms

In this chapter, existing platforms and services are examined to gain a better understanding of the instruments people use when interacting with music. Additionally, this chapter provides insight into the potential features that could be implemented in the resulting application.

Firstly, streaming services are discussed, as the main aim of the thesis is to create a system for music streaming. Then, other music-related services (e.g., forums) are presented, as they offer alternative possibilities for engaging with audio content that may be beneficial for the future platform. Lastly, platforms such as YouTube and TikTok are examined, because the survey has shown that they are a popular way to interact with music 3.1.

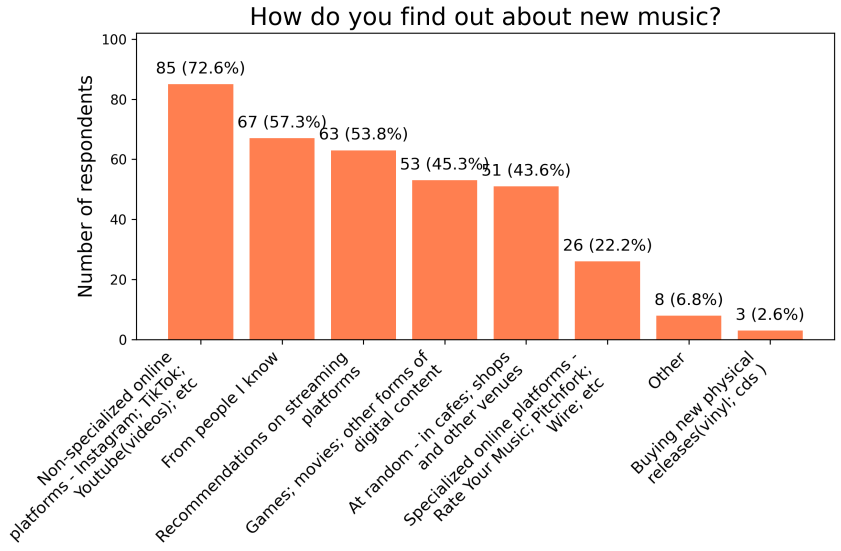

Figure 3.1: Music Discovery Methods Popularity

3.1 Streaming Services

As already discussed and illustrated in Figure 2.4, and further confirmed by the study of the International Federation of the Phono-

graphic Industry [6], the most common way of music consumption and discovery is through streaming services. Although many options are available, this section only considers those that are both popular and have unique elements. The descriptions, rather than offering a general portrayal of each platform, will focus on the service-specific social features.

3.1.1 Spotify

One of the most prominent social features on Spotify is the 'Spotify Jam' feature [7]. It lets people create a collective song queue that is then synchronized among all connected users. Moreover, volume, the order of songs, and other playback aspects can be controlled individually by each user.

Another notable feature is 'Friend Activity' [8], which shows what the people the user follows are currently listening to.

The 'Listening Parties' feature also deserves a mention: these are live chats with a limited capacity that can be joined for a short time when new music is being released [9, 10]. This somewhat brings the offline collective music discovery experience to the online world.

During the implementation of this thesis (the text was initially written in spring–summer 2025), Spotify has also added support for user-to-user chats (from September 2025) [11]. This further supports the claim that social features are in demand on music platforms.

3.1.2 VK Music

VK Music is a music service integrated into the VK social network. Consequently, it is possible to send songs and playlists via private messages or add them to posts in groups. VK Music also provides an 'Updates' tab, which is populated by the audio materials that any followed user or group has added to their 'Liked Feed' [12].

3.1.3 SoundCloud

SoundCloud is one of the few platforms that lets its users leave comments and reactions on songs and playlists [13, 14]. In addition, each

user has a feed consisting of personal uploads and reposts of others' content [15], which is visible when visiting their profile.

3.1.4 Bandcamp

Every Bandcamp user has a profile with four tabs: 'collection', 'wishlist', 'followers', and 'following'. The most interesting feature is under the 'following' tab: users can subscribe to available genres and then specify the ones they want to showcase to others viewing their profile. The 'wishlist' tab is also noteworthy, as Bandcamp's model blends streaming with traditional purchases of individual releases, and in this tab, the users can showcase what they are looking forward to listening to in the future.

This concludes the streaming services section. Services that are focused on music, but not on the streaming itself, are discussed next.

3.2 Forums, Blogs, and Other Music Media

Different resources are gathered under these umbrella terms, from professional music review websites to amateur and personal pages. Again, only widely used services that offer something noteworthy are listed.

3.2.1 Rate Your Music

> "Rate Your Music is one of the largest music databases online. It is an incredible tool that can help you find and learn about new music to listen to." [16]

This website is one of the biggest platforms with user-created content. Every member can write reviews and set ratings for music releases, contribute to the extensive 'wiki' consisting of genres, thematic lists, and charts, and connect to other members of the forum. However, the content is still moderated: any post must be properly formatted and adhere to the guidelines in order to appear on the website.

3.2.2 2step.ru

2step.ru is one of the better representatives of "old school" forums that is still active. This is a country-specific resource, and as a result, it provides not only the regular forum and article sections but also includes elements that are usually absent from larger portals. For example, a section about upcoming parties, a page dedicated to local and upcoming DJs, and an ongoing list of recorded radio shows [17].

3.2.3 last.fm

Last.fm is a music discovery and social networking service. Notable social features include friend lists, public groups for discussion, and the ability to comment on artist and track pages [18]. Additionally, the 'Events' section aggregates concert listings, allowing users to indicate that they are interested and to add comments under each event post, bridging online listening activity with real-world music events.

This section has introduced a couple of possible features not previously seen in the listed streaming services. The following section examines platforms that are not primarily focused on music but influence how people engage with it.

3.3 General-Purpose Platforms

As mentioned earlier, one of the most interesting results of the survey is the high number of people discovering music on platforms that are not focused on music. Services like Instagram, YouTube, and TikTok, which are primarily used for sharing videos and other user-generated content, have also become closely tied to how people find music and listen to it. In recent years, these platforms have started to promote music more directly by allowing users to embed tracks within their platform-native content.

Instagram, for example, added a special audio tool that lets users include music in their short videos ('Reels' [19]), helping songs reach more people through visual content.

YouTube automatically identifies songs used in videos and displays them in the description, making it easier to find and listen to them.

TikTok has had an even bigger impact: many songs have gone viral and later became hits in charts and streaming apps because people use them in popular videos (these are often called 'TikTok Hits').

This shows that big platforms are not just reacting to users' interest in music, they actively help promote it by connecting songs with content that people want to watch and share.

An integration with such services can be a big plus for the platform. A possible feature could be a badge stating "This track was in a TikTok user Alice123 sent you today!".

Summary

The table 3.1 summarizes the comparison between the features mentioned in the sections above.
Difficulty suggests the possible complexity of implementation.
Mode assumes the typical context in which the feature operates.
Social Interaction Level indicates user involvement when using the feature.

Table 3.1: Comparison of Service-Specific Social Features

Feature	Interaction Level	Mode	Difficulty
Chat	Very High	Pair	High
Spotify Jam	High	Group	Very High
Forums/Wiki	High	Group	Medium
Personal/Friends Feed	High	Individual	Low
Song/Playlist Comments	High	Group	Low
Spotify Listening Parties	Medium	Group	Very High
Embedding Music Into Posts	Medium	Group	Medium
Friends' Listening Activity	Medium	Pair	Low
Following Genres Display	Low	Individual	Medium
Real-Life Events Section	Low	Group	High
Music Wishlist	Low	Individual	Low
External Services Integration	Low	Pair/Group	High

Overall, it is clear that numerous music-related platforms with social features built in are available across the Internet. However, services that focus on the actual audio delivery often support just a subset of the possible features. This, in turn, forces users to use multiple platforms in order to meet their needs. This project aims to bridge that gap, particularly by enhancing users' potential for social interaction.

Based on the research done in the previous and current chapters, these social features were chosen for implementation, as they provide a good balance between the complexity of implementation and the level of social interaction:

- Personal/Friends Feed.
- Song/Playlist Comments.
- Friends' Listening Activity.
- Chat.

These and further requirements for the application are listed in the next chapter **Chapter 4**.

4 Specification

In this chapterm the general outline of the application is specified via functional and non-functional requirements. The individual requirements are constructed based on the 'MoSCoW' criteria [20].

4.1 Important Terms

Below is the list of important terms that appear throughout the requirements section and further chapters:

- **Anonymous User** – A User who is not yet registered in the system or has not logged in.
- **User** – An object containing information such as email, display name and other User parameters.
- **User Profile** – A Frontend-only abstraction which shows Base User information.
- **Artist** - A User with additional privileges, for example, Song or Collection creation.
- **Follower/Followee** - A relationship between a User and another User where one subscribes to another.
- **Friend** - A Followee and Follower who are mutually subscribed to each other.
- **Song** – An object representing a processed audio file with additional metadata such as 'title' or 'creation date'.
- **Collection** – Represents a container that holds multiple Songs.
- **Playlist** – A Collection consisting of pre-existing (already uploaded) Songs.
- **Album** – A Collection consisting of newly uploaded Songs.
- **Playback** – A process during which the User is receiving or manipulating audio data.

- **Playback Item** – A Song or a Collection.
- **Playback Queue** – A User-specific list of Playback Items used to determine the next active Playback Item.
- **Playback Device** – A device on which the app is used.
- **Friend Activity** – A display of the Songs currently listened to by the User's Friends.
- **Publication** – Textual User input.
- **Attachment** - Non-textual User input. For example, a Song attached to a Publication.
- **Reply** – The same as Publication, but created in relation to another, 'parent' Publication.

4.2 Functional Requirements

Must Have

- The system shall support User and Artist models.
- The system shall allow Anonymous Users to create a User model and log in to the system using an email and a password.
- The system shall allow Users to change provided information (name, bio, avatar, etc.) after the model creation.
- The system shall support over-the-net audio Playback.
- The system shall provide a way to control the audio Playback. That includes shifting the Playback backward and forward and stopping the Playback.
- The system shall provide a way to display Playback Items uploaded to it.
- The system shall limit content presentation based on a specific User. That includes Playback Items, User Profile, User Playback Devices, and other related items.
- The system shall support the creation of Playback Items. Playlists shall be created by all Users. Albums shall be created by Artists only.
- The system shall provide a Playback Queue and the ability to add or remove Playback Items from it.
- The system shall make it possible to add new Playback Devices and switch between them.
- The system shall provide a way for Users to add Publications (comments) under Collections.
- The system shall provide a personal Publication system (posts).
- The system shall provide a way for Users to create relations to other Users.

- The system shall have a way to search and filter Playback Items, Users, and Artists.

Should Have

- The system shall support adding Attachments, such as Songs or Collections, to Publications.
- The system shall provide a notification system. It shall show information about Users who followed the target User, new Reply Publication, etc.
- The system shall make it possible to filter Playback Items/Publications based on the User's relations to other Users.
- The system shall provide chat capabilities.

Could Have

- The system shall provide an Artist interface that allows Artists to see statistics related to their content.

4.3 Non-functional Requirements

Must Have

- The system shall provide access to its contents only to authorized Users. The only exceptions shall be the 'Log In' and 'Sign Up' pages.
- The system shall make unauthorized access to data as complex as possible, so that even in the case of misconfiguration of access rights, the data will be hard to access.
- The system shall store uploaded content securely.
- The system shall store and transport sensitive User information safely. I.e., it should use safe transport protocols, such as HTTPS and WSS.
- The system should be resistant to the most common attacks (CSRF, XSS, etc.).
- The system shall, in case of unexpected failure, remain rendered on the screen and show the appropriate non-technical message to the User.
- The system shall render newly available information without User input. That includes updates to Publications, Playback Items, Playback Queue, and other related items.

Should Have

- The system shall provide a responsive search method that would react to dynamic User input.
- The system shall synchronize Playback across multiple devices.

The next chapter presents the high-level technology choices used in the implementation.

5 Implementation Planning

Before the actual implementation could begin, it was necessary to choose the languages, frameworks, and other technologies that would shape the overall architecture of the application. This chapter discusses the following areas: Audio Processing and Playback, Authentication, Server–Client Communication, Backend, and Frontend.

5.1 Audio Processing and Playback

Audio Processing and Playback are the most important aspects of the application and are researched first. There are four main approaches used today: Basic Download, Progressive Download, Streaming, and Peer-to-Peer.

- **Basic Download**
  Basic Download works by downloading the entire file before the playback begins. Only when the data transfer is complete can the user start the playback.

  This method poses several limitations: for high-quality files, it typically requires significant storage space and time to download. Additionally, if the user's connection is unstable, it can make the application, which relies on playback, unusable. In cases of poor network conditions, the situation can be partially improved by providing multiple representations of the same file, such as a highly compressed version alongside a lossless one. However, after the download has started, the system is unable to adapt to the changing network state. In other words, the user is tied to the version of the file that has started to download and cannot benefit from, for instance, switching from a slow mobile network to a fast home Wi-Fi connection.

- **Progressive Download**
  With Progressive Download, it is possible to download only portions of the file, meaning the client is not restricted in playback until the whole file is on their system. This is usually implemented via a combination of HTTP byte range requests and

audio container metadata (such as MP4 indices or MP3 frame headers) that tell the client which byte ranges map to which time segments of the audio. [21] Nevertheless, *variable bitrate* is still an issue: switching between multiple file representations is fragile, since different encodings may not be perfectly aligned, and extra cross-file metadata would be needed to coordinate them. Additionally, *data storage* is problematic as well, since substantial portions of each file still have to be cached locally on the client device.

- **Streaming** [1]

  With audio Streaming, data is transmitted over the Internet in real-time without the need for the entire file to be downloaded. This is achieved by splitting the audio into short segments (chunks) with varying quality and providing a file called a manifest that lists them. That is how modern Streaming protocols support adaptive playback: the client can choose what chunk to get next based on the manifest information. [22] As a result, different bitrates and codecs can be selected based on the network conditions or device capabilities, providing an optimized playback. In addition, the client keeps only a small buffer of segments in memory and discards those that have already been played, largely fixing the issues that download methods had.

- **Peer-to-Peer**
  In P2P audio transfer, audio data is shared directly between users' devices rather than being served from a central server. Each user acts as both a client and a server, sending data to and consuming audio from other users in the peer-to-peer network. This method can reduce server load and improve access speed, but it relies heavily on user participation and network conditions.

Streaming was chosen as the playback method for this thesis. Based on the research, Basic Download was deemed too impractical, and

1. In its traditional usage, 'Streaming' means real-time media delivery over persistent transport protocols such as RTMP, where the server pushes a continuous encoded stream to the client. In this section, the term is used to refer to HTTP-based segmented Streaming (e.g., HLS, DASH).

Progressive Download in its simple form was also found to suffer from the same limitations, namely the inability to adapt to changing network conditions during playback and the need to cache large portions of the file on the client device. In a more advanced form, Progressive Download could approximate adaptive playback, but this would effectively behave like Streaming while adding several drawbacks, such as higher implementation complexity, since a custom solution for playback adaptation would need to be written.

P2P is not suitable in the current case because the files must be on users' machines, which could lead to various problems: from delays for clients located far away from the source of the audio to slow playback speeds for files with high demand and low supply.

As stated earlier, Streaming allows for smooth playback under different network conditions and reduces the memory impact a service has on the user's device. Additionally, its integration into the service, as will be demonstrated later in **Chapter ??**, is straightforward.

The next key choice would be the authentication method, as it can significantly influence both the design of the Frontend and Backend parts.

5.2 Authentication

There are many different methods of authentication available, but only those suitable for web applications are considered: Basic Authentication, Session-Based Authentication, Token Authentication, and Third-Party Authentication (OAuth2).

- **Basic Authentication**
  Basic Authentication sends a username and password with each request in the HTTP `Authorization` header, typically Base64-encoded. [23]

  The browser then usually caches those credentials after the first successful login and automatically attaches the header to subsequent requests. [2]

2. This is not usually clearly stated in the browser documentation. However, for example, in the case of Mozilla this is implied in bug tracker tickets and user documentation [24] [25].

Nonetheless, it has multiple problems, even if used over HTTPS. Since Base64 is not an encryption algorithm but only an encoding, the header can be easily decoded back. And, since the Authorization header is sent with every request, it may appear in logs or telemetry, especially after HTTPS is terminated (e.g., after reaching the internal proxy).

Moreover, there is no protocol-level notion of *logout* or per-device sessions. This means that either the Frontend must use workarounds that bypass this limitation, for instance, purposely sending a request with a manually configured `Authorization` header that has the wrong credentials, or the user has to clear their browser cache [24].

- **Cookie-Based Session Authentication**
  Session authentication makes it possible to identify users without sending their credentials on every request. Initially, the user must log in with their credentials in the same manner as for Basic Authentication. But, instead of the browser caching and reusing the credentials, the server that processed the request creates a *session* record (a data structure holding relevant user information, such as ID, roles, etc.) and instructs the browser to set a cookie with the session ID. This ID is later sent on subsequent requests instead of the user credentials and is used to identify the logged-in user. [26]

  Compared to Basic Authentication, this enables server-side logout and revocation (by deleting or invalidating sessions), per-session security policies such as device binding and inactivity timeouts, centralized control of active logins, and a simpler implementation of features that depend on temporary state.

  The main trade-off is the need for a session store, typically implemented using a relational database or an in-memory key–value store, such as Redis [27]. This introduces additional operational complexity, infrastructure dependencies, and potential bottlenecks if not configured correctly.

- **Token Authentication**
  Token Authentication replaces raw credentials with a token that

can be presented on each request instead of the login and password. The token is an opaque value issued by the server and typically sent in the `Authorization` header (usually under `Bearer` identifier).

Unlike cookie-based sessions, the server does not look up session records. Instead, it validates the token on each request, for example, by checking a signed token with a shared secret or public key, and derives the user identity and permissions from the token itself. This removes the dependency on an external session store and can simplify scaling, as any instance that knows the verification key can accept requests.

Token Authentication fits well for clients that are not browsers, such as command-line tools, mobile applications, and IoT devices, where sending cookies is inconvenient. It is also commonly used for public APIs and distributed systems, where independent services need to authenticate to each other without sharing a central session store.

Nevertheless, Token Authentication has an important trade-off: tokens are hard to revoke. Once issued, the token becomes invalid only when its expiry time has elapsed. It is also possible to keep a *revocation list* and check every request against it, but that adds state, coordination, and latency. As a result, if an attacker obtains a valid token, they can usually use it until it expires.

- **Third-Party Authentication (OAuth2)**
  Third-Party Authentication delegates user login to a third-party identity provider (via the OAuth 2.0 protocol), such as Google, GitHub, and others. [28] Instead of handling passwords directly, the application trusts this provider to authenticate the user and assert their identity.

  In the standard authentication flow, the application redirects the user to the provider's endpoint, where the user logs in. The provider then redirects back with an authorization code, which the application exchanges for tokens that can be later used to obtain user information. Using that information, the application can then create internal user models with the provided identity.

Thus, this approach offloads password storage, multi-factor authentication, and account recovery to the provider. However, it should not be the only authentication method the application provides, since it requires the user to have an account with one of the providers.

Based on the characteristics of the authentication methods, it was decided to use Session Authentication. Compared to Basic Authentication, it avoids sending user credentials with every request, supports explicit logout, per-session security policies, and extra session-related data. Compared to Token Authentication, sessions offer simpler revocation, do not require managing tokens, and are sufficient for a web application that does not need stateless or cross-service authentication. Third-Party Authentication based on OAuth 2.0 can be added on top of this design.

Another important concept that must be addressed is the server-client communication.

5.3 Server-Client Communication

Some of the features outlined in **Chapter 4** (such as immediate content updates, chat, and playback handling) require bi-directional communication. Hence, a method for communication from the server to the client must be implemented.

Because this is a web application, techniques such as Webhooks, gRPC, Pub/Sub, and others that are not directly supported by browsers are not discussed. Traditional Polling is also not considered due to its performance limitations and inefficiency. P2P techniques (e.g., WebRTC) are not listed because the application's architecture is client-server, not peer-to-peer.

This thesis focuses on four commonly used methods for server-client communication: Long Polling, Server-Sent Events, WebSockets, and Push API.

- **Long Polling**
  Long Polling is a technique where the client sends a request and waits until the server responds, holding the connection open until a response is received (or a timeout occurs). Once

a response is received, the client immediately issues another request, therefore listening to server messages continuously. [29] While this method works in environments where server-to-client messages are rare, it is unsuitable for high-throughput scenarios (especially for frequent and small payloads), as it introduces a significant overhead by repeatedly opening and closing HTTP connections. [30]

- **Server-Sent Events (SSE)**
  SSE is a one-way communication protocol that allows the server to send data to the client over a single, long-lived HTTP connection.

  Firstly, the client establishes the connection via an HTTP request, to which the server answers with a response that has its *content type* set to `text/event-stream`. The server can then reuse that open connection to send an unlimited number of responses (events). [31] Compared to Long Polling, SSE does not need to open a new HTTP connection for every update, thereby reducing the number of requests. At the same time, it is still based on plain HTTP, so it works in most browsers and fits well into networking setups.

  However, this protocol also requires additional connection handling logic: the proxy and the server must keep track of all connected clients. Moreover, special handling on the Backend must be added in order to properly route messages. [32]

- **WebSockets**
  WebSockets, in comparison to SSE, provide full-duplex communication over a single TCP connection. This is achieved by firstly creating a standard HTTP connection and then upgrading it to the WebSocket protocol via the WebSocket Protocol Handshake. [33]. This method allows messages to be sent both ways at any moment and is optimal for applications that require frequent (i.e., multiple times per second) two-way communication. [30]

  However, adding WebSockets to an application also requires additional infrastructure and logic for managing connections and message routing. In addition, WebSocket messages do not

follow the normal HTTP request-response model. They cannot be cached by standard HTTP caches [34] or CDNs, and they do not benefit from built-in features such as idempotent retries.

- **Push API**
  The Push API is a specialized web interface that allows a server to send messages to the browser without a preceding request from the client. [35] After the user grants permission, the browser creates a *push subscription* and exposes a special endpoint that the application server can use to send data to later without additional negotiation. [36] When such a request is received, the browser validates it and queues the message for processing by a special background script, known as a *service worker*. [37] Each domain (website) can register its own service worker, which is typically dormant and is only started when events such as push notifications need to be handled.

  While this mechanism is useful for notification systems, where short delivery time is not critical, it is not suitable for interactions that require an immediate response, such as updating the user's current playback state. Messages can be delayed inside the browser because multiple sites may have pending push messages to process, and the browser is free to schedule or throttle service worker execution and limit the resources available to it. As a result, push notifications typically have higher and less predictable latency compared to direct communication protocols, such as WebSockets. [35]

Based on the requirements outlined in **Chapter 4**, at least three features would require fast server-to-client communication:

- **Playback state synchronization.** The primary device would need to transmit playback updates to the server approximately once per second. The server would then propagate the synchronized playback position to all other active devices to ensure consistency.

- **Real-time typing indicators in chat.** The client would frequently send short updates to the server while the user is typing. The

server would then relay these events to the other chat participant, displaying the current typing activity in real-time.

- **Immediate screen updates on newly created content.** The screen would need to update when, for example, new Publications are added on an open Feed. For this to feel responsive, the server will send the new content to the client without requiring prior new message requests.

The initial implementation used SSE, but it proved to be less suitable even with a small number of clients. That is why WebSockets are used instead, as they reduce the overhead of client-to-server communication for messages coming in rapid succession.

At this point, the core technologies that could influence the choice of languages and frameworks are known, and the Backend is discussed in the subsequent step.

5.4 Backend

5.4.1 Language

Three languages were considered for the Backend part of the application: Python, Java, and JavaScript (Node.js). All three can be used to build a web Backend, so the decision was mainly about how much time the implementation would take and how familiar the author is with the language and its tools.

Java offers strong static typing and higher performance compared to Python and JavaScript. [38] It also has mature frameworks such as Spring Boot and good tooling support in common IDEs. However, due to the language rules and syntax, it usually requires more boilerplate code and configuration for even simple endpoints. In addition, the author has limited experience with Java web development. Using Java would first require learning and testing the framework stack, which would reduce the time available for designing and evaluating the application itself.

JavaScript with Node.js would make it possible to use the same language on both the Frontend and the Backend, and it has good support for real-time communication while being the second fastest language among the three. [39, 38] It is also widely used for Backend

services, with numerous libraries available for APIs and WebSocket-based real-time features, making it a good fit for this application in theory. Nevertheless, as with Java, the author has less experience with Node.js on the server side. Setting up a development stack, choosing and integrating the necessary libraries, and validating the design would again take a noticeable amount of time.

Python, in contrast, is the language the author knows best. It has mature web frameworks such as Django, FastAPI, and Flask, as well as good library support for common Backend tasks: authentication, database access, caching, background jobs, etc. While Python is slower in raw CPU benchmarks than Java or Node.js [39, 38], real-world web applications are usually limited by network and database latency rather than by pure CPU computational power. For an application that is primarily I/O-bound and uses external programs (i.e., processes) for CPU-bound tasks (such as audio processing), the performance provided by a Python Backend is sufficient and can be improved further by adding more worker processes or server instances if needed. Therefore, choosing Python does not introduce a practical performance bottleneck for the expected load.

Taking these points into account, Python was chosen as the Backend language. The choice is primarily made due to the author's prior experience and the limited time frame of the thesis.

5.4.2 Python Framework Comparison

As mentioned previously, there are currently three widely used frameworks available for Python: FastAPI, Django, and Flask.

- **FastAPI**
  As the official website states:

  > 'FastAPI is a modern, fast (high-performance), web framework for building APIs with Python, based on standard Python type hints.' [40].

  Its main emphasis is compatibility with Python's `asyncio`, which makes it much easier to write asynchronous code in comparison to the other two frameworks. In addition, it integrates well

with Pydantic [41] making input data validation straightforward. Moreover, routing/middleware does not require lots of configuration. [42]

However, FastAPI is relatively new and simple. It can work well for small or highly custom systems, but for standard setups, it lacks many convenience wrappers. For example, the WebSockets integration does not provide connection lifecycle management or message broadcast capabilities. [43] Additionally, no predefined templates are available for basic CRUD operations; therefore, they must be implemented manually, which adds unnecessary complexity.

And lastly, despite the steady growth of FastAPI usage, the community support is still not as wide as for Django or Flask. [44]

- **Django**
  Django is a 'batteries-included' [45] framework in the sense that it has tools for the most common web development tasks. It provides advanced user session management, convenient CSRF and XSS protection integration, flexible built-in file storage abstractions, and integrated handling of media files (images, audio, etc.). Support for REST APIs can be added through `django-rest-framework` [46], which provides generic views, serializers, and routers that make the implementation of CRUD endpoints quick with minimal custom code on top. Real-time functionality can be implemented using `django-channels` [32], an official Django package that adds connection lifecycle management, group-based message broadcasting, and integration with Django's authentication and session systems. As the documentation states, one of the main advantages of this framework is the time savings it offers. [47]

  Nonetheless, some notable drawbacks include its opinionated design choices, which can make it hard to add custom logic. Furthermore, the size of the resulting codebase can be larger because the framework ships with many modules that may remain unused, while still occupying storage space (for example, RSS integration or server-side rendering machinery). Finally, the most significant disadvantage is its limited support for asynchronous

processing [48], which often requires the use of external task queues, such as Celery [49], for long-running or background operations.

- **Flask**
  Flask is also a lightweight Python web framework. [50] It provides only a small core with routing, request/response handling, and templating, while most other functionality is added through extensions. This modular design is suitable mainly for small services or prototypes with a limited and clearly defined scope.

  However, for typical JSON-based APIs, FastAPI offers similar functionality with fewer dependencies and less setup, while Flask usually requires several additional libraries and manual configuration to achieve the same result. At the same time, Flask, like Django, is based on the synchronous WSGI interface. WebSocket support is typically added via additional tools, such as SocketIO [51], which requires the Frontend to use the same library instead of the standard WebSocket API, introducing unnecessary coupling. As a result, Flask effectively combines the disadvantages of the other two frameworks: it lacks Django's integrated ecosystem and does not provide FastAPI's modern asynchronous features. In the context of this application, it does not bring any clear advantage over Django or FastAPI.

Summing up, of the three frameworks, Django is the most suitable one. For a music streaming platform, its media handling capabilities, WebSocket support via `django-channels`, and CRUD endpoint abstractions of `django-rest-framework` lead to a better developer experience, shortening the overall implementation time. In addition, Instagram has proven that Django can handle big loads, hence, its WSGI reliance should not be a significant drawback. [52]

Having established the Backend framework, the next step was to choose the database.

5.4.3 Database

The Django framework supports several relational databases: SQLite, PostgreSQL, MySQL/MariaDB, and Oracle. Oracle is excluded due to

its commercial license [53]. Otherwise, for a project of this size, any of these options would be fast enough, because the expected number of users and the amount of stored data are not significant. Therefore, performance is not the primary factor in choosing a database.

Instead, the choice is based on the available features and how well they are integrated with Django. In this application, two requirements are especially important: full-text search and UUID primary keys (see **Chapter ??** for details). And for these requirements, PostgreSQL is the most suitable option.

First, PostgreSQL offers good full-text search support and additional tools for fuzzy matching, such as trigram indexes. [54] As a result, it can be used as the base for full-text search instead of a separate service like Elastic or Sphinx.

Moreover, Django exposes these features through the `django.contrib.postgres` module, which simplifies the creation of full-text and trigram-based queries directly from Python code. [55, 56]

Second, PostgreSQL has native support for UUID values via a dedicated `uuid` column type. [57] This type stores UUIDs in a compact binary form and allows efficient indexing and comparison. [3] In contrast, in many other databases UUIDs are usually stored as plaintext (e.g., `CHAR(36)`), which is less space- and index-efficient. Django's `UUIDField` also maps cleanly to PostgreSQL's native type, so UUID-based identifiers can be used without extra configuration.

Lastly, with the database chosen, the Frontend application type, language, and framework had to be selected.

5.5 Frontend

5.5.1 Application Type

Modern web applications are typically built either around server-side rendering (SSR), where most HTML is generated on the server, or around client-side rendering (CSR), where most HTML is generated in the browser using JavaScript. A single-page application (SPA) is an application that keeps a single HTML document loaded and updates

3. If using time-based UUID types, e.g., UUIDv7

the user interface dynamically on the client, usually relying on CSR, sometimes combined with SSR for the initial page load.

In an SSR setup, frameworks such as Django generate most of the user interface on the server and send complete HTML pages (or partial updates) to the browser, often only requiring small pieces of JavaScript for interaction. [58] In contrast, an SPA typically fetches JSON or other data from the server and, based on it, renders and updates the user interface directly in the browser. [59]

The research for the Frontend began by deciding which of these approaches to use. For many web applications, rendering HTML on the server reduces client-side complexity, avoids duplicating logic between browser and Backend, and works reliably across a wide range of devices and browsers, making it a simpler and more robust default choice.

However, in this project, the application must maintain a lot of state directly in the browser: the current playback position, the playback queue, the volume level, the chat state, etc. These values change frequently and in small steps, sometimes several times per second, and require immediate feedback for the user. Moreover, the user interface must respond to events sent by the server over WebSockets, such as playback updates or chat messages. A purely server-rendered approach with full page reloads or round-trips for each small change would be unsuitable for this kind of highly interactive, real-time interface and would complicate state handling across devices.

Because of these requirements, the Frontend is implemented as a client-side rendered single-page application (SPA).

5.5.2 Language

Currently, there are many libraries and frameworks in different languages that can be used to write web interfaces; for example, it is possible to create them even in low-level languages, such as C [60] or Rust [61]. Nevertheless, JavaScript remains the industry standard because it is still the only general-purpose language directly supported by browsers. It means that JavaScript is the only language that can be executed natively by the browser's runtime, without the need for previous pre-processing (e.g., compilation to WebAssembly). This

simplifies the development, build, and deployment process, and for these reasons, the client application is implemented in JavaScript.

5.5.3 JavaScript Framework Comparison

Modern JavaScript development is usually based on one of three libraries/frameworks: React, Vue, or Angular. [62] Each of them takes a different approach to structuring Frontend applications and has its own advantages and disadvantages.

- **React**
  React is a JavaScript library focused on building user interfaces through a component-based architecture. [63] It is defined by its declarative programming style, where the developer describes what the UI should look like for a given state, and React handles the updates. [64] Internally, React maintains a virtual representation of the DOM and only applies the minimal necessary changes to the real DOM, which helps keep many updates efficient without the developer managing manual interface update operations.

  One of the main strengths of React is its ecosystem. It has detailed documentation, numerous learning resources, and a large number of open-source packages. For instance, it integrates well with various state management tools and data-fetching libraries. Another advantage of React is React DevTools, which allows developers to inspect the currently rendered React elements and state directly in the browser, improving the debugging experience. [65] In addition, React is widely adopted and backed by a large community, so examples of real-world applications and solutions to common problems are easy to find. [66]

  At the same time, React does not provide many features. It does not include routing, form handling, or data-fetching solutions in its core. As a result, these features must be added by the developer via separate libraries. This flexibility is useful, but it also means that the developer has to make more decisions and ensure that the chosen packages work well together.

  Another point is performance optimization. For simple interfaces, React is usually fast, but in larger projects, “naive” code

can cause many unnecessary re-renders, leading to average performance compared to other frameworks. [67]

- **Vue**
  Vue, in comparison to React, is a framework: it includes many standard features out of the box, such as routing and form handling. [68] It offers a template-based syntax and single-file components, which combine markup, logic, and styles in a single file. This structure can make it straightforward to start new projects, since less setup is needed.

  The main drawback of Vue is its adoption compared to React and Angular. [62] While it has an active community, there are fewer third-party packages, fewer example projects, and fewer guides for advanced use cases. In the context of this thesis, where, as mentioned earlier, development time is a key factor, this can mean additional work, because features that exist as mature packages in other ecosystems may need to be implemented manually or with less proven tools.

- **Angular**
  Angular is a full-stack framework for building web applications. [69] Like Vue, it provides many tools out of the box, including routing, form handling, and a dependency injection system. It is built on top of TypeScript and uses strongly typed code and clearly defined architectural patterns. This can lead to more structured and robust code in large projects with many developers.

  In addition, Angular provides built-in tooling and conventions for project setup, templates, and asynchronous data handling. This makes it easier to follow a single, consistent approach across the codebase instead of combining multiple separate libraries.

  As can be seen, Angular is aimed towards bigger, enterprise-oriented projects. While they can benefit from the structure and order that Angular brings, in smaller applications, the extra code required to create and maintain the project can be a significant disadvantage, especially in the case of this thesis.

While all three frameworks offer modern development patterns and are suitable for production, React was chosen as the Frontend framework for this project. It is mature, widely used, and has an extensive ecosystem of libraries for routing, state management, and UI components. Its small core does not impose a strict application structure, which is convenient in smaller projects. Together with the available debugging tools, this makes it possible to iterate on the user interface quickly and to integrate real-time features such as playback synchronization in a straightforward way.

5.6 Summary

The platform follows the Model–View–Controller architectural pattern [70] and is organized into three logical layers:

- **Model**
  The Model layer is responsible for representing and manipulating the application data. In this project, it consists mainly of the Django models with entities such as users, songs, playlists; and the data processing logic, for example audio conversion or data querying.

- **View**
  The View layer is what the user interacts with directly. It presents the data from the Model and turns user actions into requests that can be processed by the rest of the system. Here, the View is implemented as a single-page application in JavaScript using React. It renders the interface, displays information such as playback state or friends' activity, and sends user actions (for example, play/pause commands or chat messages) to the Controller.

- **Controller**
  The Controller layer connects the View with the Model. It receives HTTP and WebSocket requests from the Frontend, applies cookie-based session authentication and access control, and then calls the appropriate Model logic. In this platform, the Controller corresponds to the Django REST API and WebSocket endpoints.

In other words, the Django models and database layer form the Model, the React single-page application acts as the View, and the Django-based REST and WebSocket endpoints play the role of the Controller that coordinates communication between them.

The next chapter, **Chapter ??**, describes the concrete implementation of these parts and how they work together to form the complete music streaming platform.

A Appendix

A.1 Survey Questionnaire

Music Consumption Study

This short form is designed to analyze the people's behaviour related to music - ways of discovering new audio pieces, the role of music in social interactions, preferred ways of consuming audio information etc.

1. How old are you?

   Mark only one oval.

   - ◯ 0-18 years old
   - ◯ 18-30 years old
   - ◯ 30-60 years old
   - ◯ 60+ years old

2. How do you mostly consume audio media?

   Tick all that apply.

   - ☐ Streaming platforms - Spotify, Apple Music, Yandex Music and others.
   - ☐ Vinyl, CD discs and other physical media.
   - ☐ Downloaded files.
   - ☐ Other: _____

3. How often do you listen to music?

   Tick all that apply.

   - ☐ Everyday.
   - ☐ Often, but not everyday - e.g. 3-5 days a week.
   - ☐ Once or twice a week at most.
   - ☐ Only on special occasions.
   - ☐ Other: _____

4. Roughly - how much songs do you listen to in a month?

Tick all that apply.

- [ ] 1000+
- [ ] 500-1000
- [ ] 100-500
- [ ] 50-100
- [ ] 10-50
- [ ] Less than 10
- [ ] Other: _____

5. Would you say that you are consistent with the music you listen to? I.e. do you mostly listen to the same artists/songs/genres?

Mark only one oval.

- ◯ Yes
- ◯ Mostly
- ◯ Rather no than yes
- ◯ No

6. How often do you listen to something new?

Mark only one oval.

- ◯ Very rarely, once a couple of months.
- ◯ Rarely, maybe a handful times a month.
- ◯ Pretty often, every week or so.
- ◯ Often, a couple of times a week.
- ◯ Every or almost every day.

7. How do you find out about new music?

Tick all that apply.

- [ ] At random - in cafes, shops and other venues.
- [ ] From people I know.
- [ ] Non-specialized online platforms - Instagram, TikTok, Youtube(videos), etc.
- [ ] Specialized online platforms - Rate Your Music, Pitchfork, Wire, etc.
- [ ] Games, movies, other forms of digital content.
- [ ] Buying new physical releases(vinyl, cds ...)
- [ ] Recommendations on streaming platforms
- [ ] Other: _____

8. Would you say that you know a lot of people with similar music taste?

Mark only one oval.

- ◯ Yes
- ◯ No

9. Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?

Mark only one oval.

- ◯ Yes
- ◯ No, I already have enough people around me with whom I can share the music I like
- ◯ No, I do not feel the need in that
- ◯ Other: _____

10. Are there any situations, when you listen to music with someone else?

Tick all that apply.

- [ ] At parties; when I'm visiting someone; in a car.
- [ ] Online via Spotify Jam, Discord bots and other similar instruments.
- [ ] Concerts, clubs, street performances.
- [ ] At work.
- [ ] Other: _____

11. How often do you listen to music with someone else?

Mark only one oval.

- ◯ Very rarely, maybe a couple times a year.
- ◯ Rarely, every one or two months.
- ◯ Often, a couple of times a month.
- ◯ Very often, every week or so.
- ◯ Almost every day.
- ◯ Other: _____

12. How often do you/your friends share/discuss music?

Mark only one oval.

- ◯ Very rarely, maybe a couple times a year.
- ◯ Rarely, every one or two months.
- ◯ Often, a couple of times a month.
- ◯ Very often, every week or so.
- ◯ Almost every day.
- ◯ Other: _____

13. How does that happen?

Tick all that apply.

- [ ] Online, sending links from streaming platforms via messengers.
- [ ] Offline, in a car, at someone's place etc.
- [ ] Via sharing the physical media(e.g. giving a vinyl to a friend for a listen)
- [ ] Other: _____

14. Question for users of streaming platforms.

What platform do you use?

Tick all that apply.

- [ ] Spotify
- [ ] Apple Music
- [ ] Yandex Music
- [ ] VK Music
- [ ] SoundCloud
- [ ] Bandcamp
- [ ] Youtube Music
- [ ] Other: _____

15. Question for users of streaming platforms.

Would you benefit, if current streaming platforms added extra social features(i.e. would you use them)? For example, chat discussions for playlists and songs, feeds with music added by the people you follow, forums, direct messages, etc.

Mark only one oval.

- ( ) I would use these features often.
- ( ) I would use these features rarely.
- ( ) I wouldn't use these features.

16. Question for users of streaming platforms.

Are there any other features you would like to see?

Bibliography

[1] *Music Streaming Market Decade Insights*. URL: https://www.businessofapps.com/data/music-streaming-market/ (visited on 11/17/2025).

[2] Peter J. Rentfrow. *The Role of Music in Everyday Life: Current Directions in the Social Psychology of Music*. 2012. URL: https://compass.onlinelibrary.wiley.com/doi/abs/10.1111/j.1751-9004.2012.00434.x (visited on 03/15/2025).

[3] *Sharing Music: Social and Communal Aspects of Concert-Going*. URL: https://ojs.meccsa.org.uk/index.php/netknow/article/view/428/250 (visited on 11/17/2025).

[4] *Google Forms*. URL: https://docs.google.com/forms/u/0/ (visited on 04/05/2025).

[5] *Spotify Popularity*. URL: https://simplebeen.com/music-streaming-statistics/?utm_source=chatgpt.com (visited on 04/15/2025).

[6] *Audio Usage Statistics 2024*. URL: https://www.ifpi.org/wp-content/uploads/2023/12/IFPI-Engaging-With-Music-2023_full-report.pdf (visited on 03/15/2025).

[7] *Spotify Jam*. URL: https://support.spotify.com/cz/article/jam/ (visited on 03/16/2025).

[8] *Spotify Friend Activity*. URL: https://support.spotify.com/cz/article/friend-activity/ (visited on 03/16/2025).

[9] *Spotify Party Announcement*. URL: https://newsroom.spotify.com/2024-09-12/fans-artists-connections-features/ (visited on 03/16/2025).

[10] *Spotify Party*. URL: https://support.spotify.com/us/article/listening-party/ (visited on 03/16/2025).

[11] *Spotify Chat*. URL: https://newsroom.spotify.com/2025-08-26/introducing-messages-a-new-way-to-share-what-you-love-on-spotify-with-friends-and-family/ (visited on 11/17/2025).

[12] *VK Music*. URL: https://music.vk.com/ (visited on 11/17/2025).

[13] *SoundCloud Comments*. URL: https://help.soundcloud.com/hc/en-us/articles/115003566008-Commenting-basics (visited on 03/16/2025).
[14] *SoundCloud Reactions*. URL: https://help.soundcloud.com/hc/en-us/articles/31039497560731-Reactions (visited on 03/16/2025).
[15] *Soundcloud Reposts*. URL: https://help.soundcloud.com/hc/en-us/articles/115003567488-Reposting-tracks-or-playlists (visited on 03/16/2025).
[16] *Rate Your Music*. URL: https://rateyourmusic.com/discover (visited on 03/25/2025).
[17] *2step.ru*. URL: http://www.2step.ru/ (visited on 03/25/2025).
[18] *LastFM*. URL: https://www.last.fm/home (visited on 05/10/2025).
[19] *Instagram Audio*. URL: https://help.instagram.com/664508917489819?helpref=faq_content (visited on 03/25/2025).
[20] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. URL: https://dl.acm.org/doi/10.5555/561543 (visited on 05/05/2025).
[21] *Adaptive progressive download based on the MPEG-4 file format*. URL: https://link.springer.com/article/10.1631/jzus.2006.AS0106 (visited on 11/30/2025).
[22] *The MPEG-DASH Standard for Multimedia Streaming Over the Internet*. URL: https://ieeexplore.ieee.org/abstract/document/6077864 (visited on 11/28/2025).
[23] *Basic Authentication*. URL: https://datatracker.ietf.org/doc/html/rfc7617 (visited on 03/19/2025).
[24] *Mozilla Caching Behaviour Bug Tracker*. URL: https://bugzilla.mozilla.org/show_bug.cgi?id=300002 (visited on 11/01/2025).
[25] *Mozilla Caching Behaviour Documentation*. URL: https://support.mozilla.org/en-US/kb/delete-browsing-search-download-history-firefox (visited on 11/01/2025).
[26] *Session Authentication*. URL: https://docs.djangoproject.com/en/5.2/topics/http/sessions/ (visited on 03/19/2025).
[27] *Redis*. URL: https://redis.io/ (visited on 12/07/2025).

[28] *OAuth2*. URL: https://datatracker.ietf.org/doc/html/rfc6749 (visited on 03/21/2025).

[29] *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP*. URL: https://datatracker.ietf.org/doc/html/rfc6202 (visited on 11/30/2025).

[30] *Comparison of Server to Client Communication Methods*. URL: https://www.diva-portal.org/smash/get/diva2:1133465/FULLTEXT01.pdf?ref=hackernoon.com (visited on 03/29/2025).

[31] *Server Sent Events*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events (visited on 05/02/2025).

[32] *Django Channels*. URL: https://channels.readthedocs.io/en/latest/ (visited on 03/19/2025).

[33] *WebSockets*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (visited on 05/02/2025).

[34] *Http Caching*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching (visited on 05/02/2025).

[35] *Push API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Push_API (visited on 05/02/2025).

[36] *Generic Event Delivery Using HTTP Push*. URL: https://datatracker.ietf.org/doc/html/rfc8030 (visited on 11/30/2025).

[37] *Service Worker API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (visited on 11/30/2025).

[38] *Java, Python and Javascript, a comparison*. URL: https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1355073&dswid=-4197 (visited on 12/07/2025).

[39] *JavaScript*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript (visited on 12/07/2025).

[40] *FastAPI*. URL: https://fastapi.tiangolo.com/ (visited on 03/18/2025).

[41] *Pydantic*. URL: https://docs.pydantic.dev/latest/ (visited on 05/01/2025).

[42] *FastAPI Features*. URL: https://fastapi.tiangolo.com/features (visited on 03/20/2025).

[43] *FastAPI Websockets*. URL: https://fastapi.tiangolo.com/advanced/websockets/ (visited on 12/01/2025).
[44] *StackOverflow 2024 Developers Survey*. URL: https://survey.stackoverflow.co/2024/technology (visited on 12/01/2025).
[45] *Django*. URL: https://www.djangoproject.com/ (visited on 03/18/2025).
[46] *DRF*. URL: https://www.django-rest-framework.org/ (visited on 03/19/2025).
[47] *Django Overview*. URL: https://www.djangoproject.com/start/overview/ (visited on 12/07/2025).
[48] *Django Async Support*. URL: https://docs.djangoproject.com/en/5.2/topics/async/ (visited on 01/30/2025).
[49] *Celery Library*. URL: https://docs.celeryq.dev (visited on 12/07/2025).
[50] *Flask*. URL: https://flask.palletsprojects.com/en/stable/ (visited on 03/19/2025).
[51] *SocketIO*. URL: https://socket.io/ (visited on 12/01/2025).
[52] *Web Service Efficiency at Instagram with Python*. URL: https://instagram-engineering.com/web-service-efficiency-at-instagram-with-python-4976d078e366 (visited on 12/07/2025).
[53] *Oracle Price List*. URL: https://www.oracle.com/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf (visited on 11/30/2025).
[54] *PostgreSQL Full Text Search*. URL: https://www.postgresql.org/docs/current/textsearch.html (visited on 12/07/2025).
[55] *PostgreSQL*. URL: https://www.postgresql.org/about/ (visited on 03/26/2025).
[56] *Django PostgreSQL Support*. URL: https://docs.djangoproject.com/en/5.2/ref/contrib/postgres/ (visited on 03/26/2025).
[57] *Postgres UUID Type*. URL: https://www.postgresql.org/docs/current/datatype-uuid.html (visited on 12/01/2025).
[58] *Django Templates*. URL: https://docs.djangoproject.com/en/5.2/ref/templates/language/ (visited on 01/30/2025).
[59] *Single Page Application*. URL: https://developer.mozilla.org/en-US/docs/Glossary/SPA (visited on 12/07/2025).
[60] *Clay Library*. URL: https://github.com/nicbarker/clay (visited on 12/01/2025).

[61] *Yew Framework*. URL: https://github.com/yewstack/yew (visited on 12/01/2025).
[62] *Framework Usage*. URL: https://2024.stateofjs.com/en-US/libraries/ (visited on 05/01/2025).
[63] *React*. URL: https://react.dev/ (visited on 03/30/2025).
[64] *How declarative UI compares to imperative*. URL: https://react.dev/learn/reacting-to-input-with-state#how-declarative-ui-compares-to-imperative (visited on 12/07/2025).
[65] *React Developer Tools*. URL: https://react.dev/learn/react-developer-tools (visited on 05/01/2025).
[66] *StackOverflow React*. URL: https://stackoverflow.com/questions/tagged/reactjs?tab=Newest (visited on 12/07/2025).
[67] *Addressing the Limitations of React JS*. URL: https://www.academia.edu/44039524/IRJET_Addressing_the_Limitations_of_React_JS (visited on 12/07/2025).
[68] *VueJS*. URL: https://vuejs.org/about/faq.html (visited on 03/30/2025).
[69] *Angular*. URL: https://angular.dev/ (visited on 12/07/2025).
[70] *MVC*. URL: https://developer.mozilla.org/en-US/docs/Glossary/MVC (visited on 03/18/2025).